

Campus Recruitment Training — Phase 2

Post-Assessment Test

Duration: **90 Minutes**

Total Marks: **100**

Platform: **Eclipse IDE / IntelliJ
IDEA**

Instructions: All questions are code-based. Write clean, readable Java/YAML/JS code as applicable. Attempt all sections. Partial credit is awarded where marked. You may write short comments to explain your logic — it helps in evaluation.

Section A — Spring Boot & Request Lifecycle

[25 marks]

Q1. Predict the output. What will print to the console when this Spring Boot application starts? Write the output in order and give a one-line reason for each. **[4M]**

```
@Component
public class MyBean {
    public MyBean() {
        System.out.println("Constructor called");
    }

    @PostConstruct
    public void init() {
        System.out.println("Init called");
    }
}
```

Q2. The endpoint below compiles and runs but always returns an empty response body. Identify the mistake and write the corrected line only. **[3M]**

```
@Controller
@RequestMapping("/api/users")
public class UserController {
    @Autowired UserService service;

    @GetMapping("/{id}")
    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User u = service.findById(id);
        return new ResponseEntity<>(HttpStatus.OK);
    }
}
```

Q3. Write a Spring Boot REST endpoint POST /api/students that accepts a JSON body with fields name (must not be blank) and age (must be 18 or above). Return 400 Bad Request with an error message if validation fails, or 201 Created on success. Write the DTO class and the controller method. **[5M]**

Q4. What annotation should be added to the transfer() method below, and why? Write the annotation and explain in 2 lines what happens if it is missing. **[3M]**

```
@Service
public class BankService {
    @Autowired AccountRepository repo;

    public void transfer(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to = repo.findById(toId).orElseThrow();
        from.setBalance(from.getBalance() - amount);
        to.setBalance(to.getBalance() + amount);
        repo.save(from);
        repo.save(to);
    }
}
```

Q5. Write a JPA entity class Employee with the following fields: id (auto-generated), name (String), salary (double), and a Many-to-One relationship to a Department entity. Show the Department entity with a One-to-Many list of employees. Use correct annotations on both sides. **[5M]**

Q6. Look at the JWT filter below. (a) Explain in 2 lines what this filter does. (b) There is one bug — the token extracted from the header will always be invalid. Find it and write the corrected line. **[5M]**

```
String header = req.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) {
    chain.doFilter(req, res);
    return;
}
String token = header.substring(6); // extract token
String username = jwtUtil.extractUsername(token);
if (username != null) {
    UsernamePasswordAuthenticationToken auth =
        new UsernamePasswordAuthenticationToken(username, null, null);
    SecurityContextHolder.getContext().setAuthentication(auth);
}
chain.doFilter(req, res);
```

Section B — JPA, SQL & Data Layer**[20 marks]**

Q7. Write a Spring Data JPA repository method using `@Query` to find all Product entities where price is less than a given value and category matches a given string. Use named parameters. **[4M]**

Q8. Explain the N+1 query problem in JPA in your own words with a simple example. What is the most common cause? Write one line of code that fixes it in a `@Query` method using `JOIN FETCH`. **[4M]**

Q9. Write a SQL query to find the second highest salary from a table `employees(id, name, salary)`. Use a subquery approach. Then write the same logic as a JPQL `@Query` method in a JPA repository. **[4M]**

Q10. Given a Student entity with fields `name`, `email`, and `city`, write FOUR Spring Data JPA derived query method names (no `@Query`) for the following: (a) find by city, (b) find by name and city, (c) find by email containing a keyword, (d) count students in a city. Write the method signatures only. **[5M]**

Q11. A developer runs the following code to load 10 orders and print their items. The **[3M]**

console shows 11 SQL queries instead of 1. What is this problem called? Write the line of code that causes it and the one-line fix.

```
List<Order> orders = orderRepo.findAll();
for (Order o : orders) {
    System.out.println(o.getItems().size());
}
```

Section C — Microservices & System Design

[25 marks]

Q12. Write the application.yml configuration and the main class annotation needed to register a Spring Boot service named product-service with a Eureka server at <http://localhost:8761/eureka>.

[4M]

Q13. What is a Circuit Breaker in microservices? Why is it needed? Write a Spring Boot method that calls an external payment-service using RestTemplate and applies a Resilience4j @CircuitBreaker with a fallback method that returns a default PaymentResponse object.

[5M]

Q14. What is an API Gateway and what are 2 benefits of using one? Write a Spring Cloud Gateway application.yml that routes requests starting with /api/orders/** to order-service and /api/users/** to user-service. Use lb:// for load-balanced URIs.

[6M]

Q15. What is Service Discovery in microservices and why is it needed instead of hardcoding IP addresses? Name the tool you used in class. Write a RestTemplate bean configuration that enables it to resolve service names automatically.

[4M]

Q16. You have a monolithic Spring Boot app with modules: UserModule, OrderModule, ProductModule. Your team wants to split it into microservices. (a) List 3 benefits of this split. (b) List 2 challenges. (c) Briefly explain how Eureka and an API Gateway work together so a React frontend can talk to all three services through a single URL.

[6M]

Section D — Testing & Docker

[20 marks]

Q17. Write a JUnit 5 + Mockito unit test for the `getById()` method below. Your test should: (a) use `@Mock` and `@InjectMocks`, (b) stub the repository to return a sample Product, and (c) assert that the returned product name equals "Laptop".

[5M]

```
@Service
public class ProductService {
    @Autowired ProductRepository repo;

    public Product getById(Long id) {
        return repo.findById(id).orElseThrow();
    }
}
```

Q18. What is the difference between `@Mock` (Mockito) and `@MockBean` (Spring Boot Test)? When would you use each? Write one example of a test class declaration that uses `@MockBean` correctly with `@SpringBootTest`.

[4M]

Q19. Write a Dockerfile for a Spring Boot application. The JAR file is named `app.jar` and is inside the `target/` folder. Use `eclipse-temurin:21-jre` as the base image. The app runs on port 8080. Then write the docker build and docker run commands to build the image as `myapp:latest` and run it.

[5M]

Q20. Write a `docker-compose.yml` with two services: (a) `db` — using the `postgres:15` image with environment variables for DB name, user, and password, and (b) `app` — using the `myapp:latest` image, mapping port 8080, connecting to the `db` service, and starting only after `db` starts using `depends_on`.

[6M]

Section E — Frontend Integration (Bonus)

[10 marks]

Q21. Write a JavaScript async function `loadUsers()` that fetches data from `GET /api/users` using the `fetch` API, checks if the response is successful (`response.ok`), and either renders a list of user names into a `<ul id="user-list">` element or shows an error message in a `<p id="error">` element.

[5M]

Q22. The React component below has a bug that causes it to fetch data in an infinite loop.

[5M]

(a) Identify exactly what is wrong. (b) Fix it. (c) Add a simple loading state that shows 'Loading...' while data is being fetched.

```
function UserList() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => setUsers(data));
  }); // <-- notice anything?

  return <ul>{users.map(u => <li key={u.id}>{u.name}</li>)}</ul>;
}
```